MODULE 5

Agents and automation



By Innovation Team

A simple request.
A bigger ask.

> **“Triage alert NW-4471 – and take care of it.”**

A chatbot can only tell you what to do. Taking care of it means looking things up and acting – that needs tools.

ANSWER ≠ ACTION



An agent is a model in a loop that can call your functions. The model only asks – your code decides what runs.

Chat (Module 3)

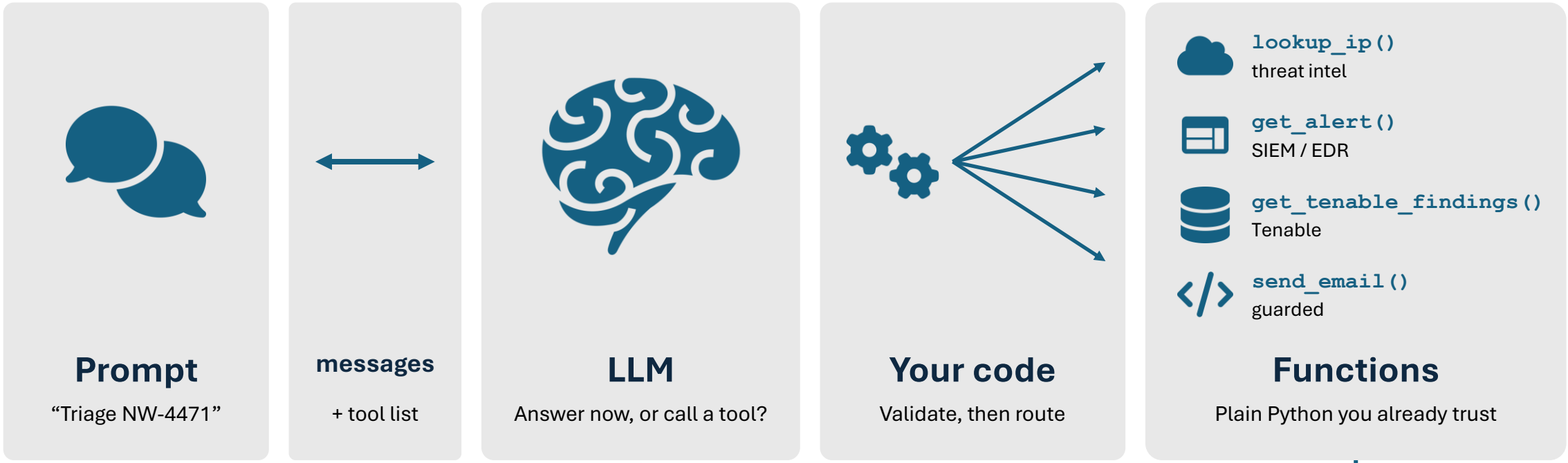
Tells you what to do

AGENT

Looks things up and acts through your tools

Tool/function calling

The model picks the function and arguments. Your code makes the call.



Tool/function calling is the ability of an LLM to request a call to an external tool, function or API, by returning the function name and arguments in a structured format. Your application validates the request, runs the call only if it is allowed, and returns the result to the model.

Tool/function calling

Navigate to the lab 05-llm directory and execute the following modules.

Execute
pip install "pydantic==2.13.5“

```
((venv) ) [opc@n8n-chi 05-llm]$ pip install "pydantic==2.13.5"
Collecting pydantic==2.13.5
  Obtaining dependency information for pydantic==2.13.5 from https://files.pythonhosted.org/packages/eb/47/c95ffc2009878c7aac0c5e08528022dcb885933252a88b5f170058014464/pydantic-2.13.5-py3-none-any.whl.
metadata
  Downloading pydantic-2.13.5-py3-none-any.whl.metadata (110 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 110.2/110.2 kB 12.6 MB/s eta 0:00:00
Requirement already satisfied: annotated-types>=0.6.0 in /home/opc/venv/lib64/python3.12/site-packages (from pydantic==2.13.5) (0.8.0)
Collecting pydantic-core==2.46.5 (from pydantic==2.13.5)
  Obtaining dependency information for pydantic-core==2.46.5 from https://files.pythonhosted.org/packages/c0/a4/eb9409ec0736e50aa70a412f16c204ed149516846912f7e6724d4c73ee53/pydantic_core-2.46.5-cp312-c
p312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata
  Downloading pydantic_core-2.46.5-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.6 kB)
Requirement already satisfied: typing-extensions>=4.14.1 in /home/opc/venv/lib64/python3.12/site-packages (from pydantic==2.13.5) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.2 in /home/opc/venv/lib64/python3.12/site-packages (from pydantic==2.13.5) (0.4.4)
Downloading pydantic-2.13.5-py3-none-any.whl (472 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 472.6/472.6 kB 29.6 MB/s eta 0:00:00
Downloading pydantic_core-2.46.5-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.1/2.1 MB 131.1 MB/s eta 0:00:00
Installing collected packages: pydantic-core, pydantic
  Attempting uninstall: pydantic-core
    Found existing installation: pydantic_core 2.50.1
    Uninstalling pydantic_core-2.50.1:
      Successfully uninstalled pydantic_core-2.50.1
  Attempting uninstall: pydantic
    Found existing installation: pydantic 2.14.1
    Uninstalling pydantic-2.14.1:
      Successfully uninstalled pydantic-2.14.1
Successfully installed pydantic-2.13.5 pydantic-core-2.46.5
WARNING: There was an error checking the latest version of pip.
((venv) ) [opc@n8n-chi 05-llm]$ python3 lab5_function_calling.py --offline
```

Steps 1–2: what goes in

python3 lab5_function_calling.py --offline · Nothing has run yet. The model only gets text.



1 · Prompt

The analyst's request, plus a system prompt that gives Nova its role and rules.

```
[1 PROMPT] what the analyst asked
  Triage NW-4471 and email a two-line
  summary to soc-lead@northwind.example.
[1 PROMPT] plus the system prompt
  You are Nova, the Northwind SOC ...
```

- Both go into the messages list
- The model never touches your systems



2 · Messages + tool list

Messages plus four tool schemas, generated from the Pydantic classes. Sent again on every step.

```
[2 MESSAGES + TOOL LIST] from the Pydantic classes:
  {"name": "lookup_ip", "parameters": {
   "properties": {"ip": {"type": "string",
                         "format": "ipv4"}},
   "additionalProperties": false}}
```

- class LookupIpArgs → this JSON, automatically
- --show-schemas prints all four

Steps 3–4: the model asks, your code decides

The LLM writes a request. Your code checks it before anything runs.

3 · LLM: answer, or call a tool?

Step 1: the model asks for the alert first. It is a request, not an action.

```
[3 LLM] step 1, 28 output tokens
[3 LLM] asked for a tool:
  {"tool_calls": [{"function": {
    "name": "get_alert",
    "arguments": {"alert_id": "NW-4471"}}}]}
```

- The model chose get_alert by itself
- On CPU each step takes 10–60 s

4 · Your code: validate, then route

Pydantic checks the shape: types, formats, unknown fields. Then guard() checks permission. Only then is the call routed.

```
[4 YOUR CODE] Pydantic: OK -> GetAlertArgs(...)
[4 YOUR CODE] guard: allowed -> route
# step 2, the model sent "203.0.113.66:443":
[4 YOUR CODE] Pydantic: REJECTED - not a valid
  IPv4 address. The function does not run.
```

- Shape: the Pydantic class · Permission: guard()
- --to soc-archive@... → guard: REFUSED

Step 5 and back: the function runs, the loop turns

Only now does real code touch SOC data. Then the result goes back to the LLM.



5 · Functions: plain Python runs

Step 3: lookup_ip runs on the IP from get_alert, after the model fixed its step-2 call.

```
[5 FUNCTIONS] lookup_ip(ip="203.0.113.66")
  returned:
  {"verdict": "malicious",
   "tags": ["C2", "Cobalt Strike"],
   "first_seen": "2026-10-28"}
```

- get_alert → host FIN-LT-022, IP 203.0.113.66
- get_tenable_findings → critical VPN RCE



Result back: the loop

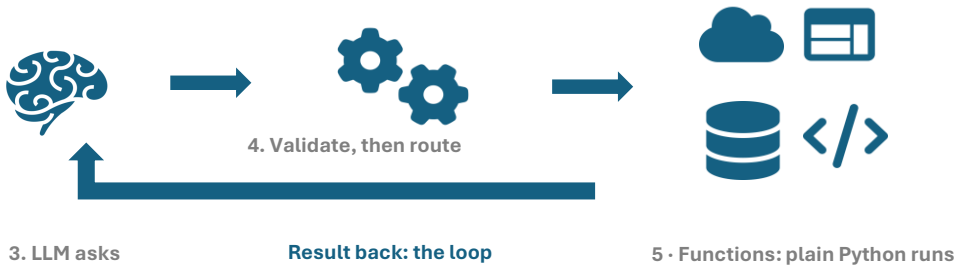
Each result, or error, becomes a tool message. The LLM reads all of them and chooses the next step.

```
[RESULT BACK] -> 4 messages    (step 1)
[RESULT BACK] -> 6 messages    (step 2: the error)
[RESULT BACK] -> 8 messages    (step 3: retry OK)
  ... 10, 12 messages         (steps 4, 5)
[3 LLM] step 6: no tool call -> it answered
```

- Errors go back too: that is how it retries
- The step budget (8) caps the loop

The output: six trips around the diagram

Step · LLM asked	Validate + Tool/Function Call	Function result
1 · get_alert(NW-4471)	OK → get_alert()	EDR beacon: FIN-LT-022 → 203.0.113.66
2 · lookup_ip(203.0.113.66:443)	**REJECTED by your code**	**never runs; error: not a valid IPv4 address**
3 · lookup_ip(203.0.113.66)	OK → lookup_ip()	malicious · C2 · Cobalt Strike
4 · get_tenable_findings(FIN-LT-022)	OK → get_tenable_findings()	Critical VPN client RCE, 41 days old
5 · send_email(soc-lead@northwind.example)	OK, approved domain	sent
6 · (no tool call)	–	Final answer: HIGH. Isolate, block, patch
Try it: 5 · send_email(soc-archive@northwind-backup.net)	**Pydantic OK, REFUSED by guard**	**never runs; the refusal goes back**



The agent loop: function calling, repeated

Slide 6 is one trip (orange). The agent repeats it until the model answers or the step budget runs out.

